Module 2: Perception Layer, Research & Design

OmniSight module doc | Prepared for Juhi Thacker | Covers roadmap Week 1, Days 5-7

1. Purpose of this Module

The perception layer lets the bot **see the page the way a person does**. Given a plain-English description such as “the Login button”, it finds the matching element with no stored selector. It also checks after each step whether the expected state was reached (the Verifier).

This module has three parts:

1. **DOM simplifier**: turns a page into a short, numbered list of visible interactive elements.
2. **Element finder**: sends that list (and, when needed, a screenshot) to a multimodal model and gets back the matching element.
3. **Verifier**: asks the model whether the page now matches the step’s `expect` text.

Done when: the bot clicks “the Login button” with no selector, and every step is verified.

2. Research Findings

2.1 Existing agents index the interactive DOM

- Browser-automation agents such as Browser-Use extract only the interactive elements (inputs, buttons, links), give each a numeric index, and let the model say “click element 14” instead of writing a selector. The result is much smaller than raw HTML.
- Stagehand follows a similar design. It crawls the DOM, keeps leaf elements with real user-facing content plus interactive elements, discards anything not visible or not active, and generates a stable XPath for each candidate. The model sees a **numbered list of independent elements**, not a full DOM, and the map from number to XPath stays on your side.
- Stagehand’s default mode **tries the DOM-only approach first and retries with an annotated screenshot only if the model fails to pick a single element**. This matches OmniSight’s rule: heal or use vision only on failure.
- An academic DOM-grounded assistant (PageGuide) stores each indexed element as an integer ID, its text, its tag type, and its on-screen bounding box. That is a good minimum schema for your simplifier.

2.2 Do not ask the model for raw pixel coordinates

- Research on GUI grounding repeatedly finds that vision-language models are weak at **outputting exact (x, y) coordinates** from a plain screenshot, but much better at **choosing a labeled option**.
- **Set-of-Mark (SoM)** prompting draws numbered marks over detected elements on the screenshot, and the model replies with a label. One recent study of multimodal agents found a very wide spread in coordinate-prediction accuracy across models, while label selection was far more consistent.
- A 2025 paper reports that converting coordinate prediction into discrete grid or mark ID selection lifted one model from single-digit to roughly 70% click accuracy on a standard benchmark. Treat the exact numbers as indicative, not guaranteed for your setup.
- **Impact on OmniSight:** your vision fallback should overlay numbered boxes on the screenshot using bounding boxes from the DOM, and ask the model to return a number. Use raw coordinates only for the desktop mode in Week 5, where no DOM exists.

2.3 ARIA snapshots are a second source

Playwright’s accessibility-tree snapshot (see the Module 1 doc) gives roles and accessible names in a compact form. It can serve as an alternative or complement to your own DOM simplifier. A sensible plan: build the simple DOM simplifier first, and compare it to the ARIA snapshot during the Week 2 stress test.

3. Design Decisions

Decision	Choice	Reason
Element list	Visible interactive elements only, numbered	Small prompts, lower cost, matches proven agent designs
Element identity	Index mapped to a locally stored locator set	Model never sees or invents selectors
Model output	JSON with `index`, `confidence`, `reason`	Easy to validate and to log for the dashboard
First attempt	DOM list only (text, no image)	Cheaper and faster
Fallback	Screenshot with numbered boxes (Set-of-Mark)	Better than asking for coordinates
Click method	Resolve the chosen index to a Playwright locator, then click	Reuses auto-wait and actionability checks
Verifier input	Screenshot plus short DOM summary plus the `expect` text	Catches both visual and structural failures

4. DOM Simplifier Design

Run a small script inside the page (via Playwright's `page.evaluate`) that collects candidates and returns a list like this:

```
[
  {
    "index": 7,
    "tag": "button",
    "role": "button",
    "text": "Login",
    "ariaLabel": null,
    "placeholder": null,
    "id": "login-btn",
    "box": { "x": 1180, "y": 24, "w": 84, "h": 36 },
    "inViewport": true
  }
]
```

What counts as a candidate

- Tags: `a`, `button`, `input`, `select`, `textarea`, `summary`
- Elements with roles such as `button`, `link`, `checkbox`, `menuitem`, `tab`
- Elements with a click handler or `tabindex`

What to discard

- Hidden elements (`display:none`, `visibility:hidden`, zero size)
- Disabled elements
- Elements fully covered by another element (check with `document.elementFromPoint` at the center)

Keep a private map from each index to several locator candidates (CSS, XPath, text, role, label). This map is what the healing module reuses later.

Mask sensitive fields: never include password values in the list. Mark them as `sensitive` and send only the label.

5. Element Finder Design

Prompt contract (text-only first attempt)

- Input: the step description, the numbered element list, and the page title and URL.
- Instruction: choose the single best match, or say none match.
- Output (JSON only): `{ "index": 7, "confidence": 0.0-1.0, "reason": "..." }` or `{ "index": null, "reason": "..." }`.

Decision flow

1. Build the element list.
2. Ask the model with text only.
3. If the answer is valid and confidence is above the threshold, resolve and act.
4. If the model returns null, low confidence, or invalid JSON, **retry once with the Set-of-Mark screenshot**.
5. If that also fails, return a classified error so Module 3 (healing) takes over.

Set-of-Mark screenshot: inject a temporary overlay that draws each element's bounding box with its index number, take the screenshot, then remove the overlay. Draw labels in a high-contrast color and avoid overlapping labels where possible.

Safety checks before acting

- The returned index must exist in the current list.
- The target must be visible and enabled.
- If the step is destructive (delete, pay, submit), pass it to the safety layer first. That module comes in Week 3, so leave a hook now.

6. Verifier Design

After each step, compare the real page with the step's `expect` text.

- **Input:** a fresh screenshot, a short list of visible headings and key text, and the `expect` sentence.
- **Output (JSON only):** `{ "met": true|false, "confidence": 0.0-1.0, "observation": "..." }`.
- **Cheap checks first:** URL change, a specific text appearing, or a dialog visible. Call the model only when a cheap check cannot decide.
- **Cost control:** run the model Verifier only on steps that have an `expect` field.
- A failed verification is treated as a step failure with the error type `EXPECTATION_NOT_MET`, so the healer can respond.

7. Model Wrapper

Put every model call behind one function, for example `askModel({ system, text, images, schema })`.

- Enforces JSON-only replies and validates them.
- Logs prompt size, latency, and estimated cost for the dashboard.
- Lets you switch providers without touching the rest of the code.
- Add a simple cache keyed by page-state hash plus description to avoid repeat calls during testing.

Because the development laptop has 8 GB RAM, use only a hosted model API.

8. Folder Layout for this Module

```
perception/
  domSimplifier.ts   # page.evaluate script plus types
  marks.ts           # Set-of-Mark overlay and screenshot
  finder.ts          # findElement(description) with text then vision flow
  verifier.ts        # verifyStep(expect)
  model.ts           # askModel wrapper, JSON validation, logging
```

Hook into the engine by replacing the plain-selector body of `resolveElement(step)` from Module 1 with a call to `finder.findElement`.

9. Build Checklist

Day 5

- Write the DOM simplifier and print the list for the Shoes Mart home page
- Check the list by eye: are the right elements present and nothing hidden?

Day 6

- Write `model.ts` and `finder.ts` (text-only path)
- Make the bot click “the Login button” with the selector removed from the workflow

Day 7

- Add the Set-of-Mark fallback
- Write `verifier.ts` and connect it after each step
- Run the full demo workflow with no selectors and record the results in the devlog

10. Test Cases

Test	Expected result
Find “the Login button” on the normal page	Correct element, high confidence
Same, after the ID is renamed	Still found
Same, after the text changes to “Sign in”	Found by role and position
Two similar buttons on the page	Picks the right one or reports ambiguity
Description of an element that does not exist	Returns null, no random click
Verifier given a wrong expectation	Returns `met: false`

11. Risks and Open Questions

- **Hallucinated indexes:** always validate the index against the list.
- **Large pages:** cap the list (for example 150 elements) and rank by viewport visibility.
- **Iframes and shadow DOM:** the first version may miss these. Note them as known limitations.
- **Cost and latency:** the text-only path should handle most steps. Track how often the vision fallback fires.
- **Open question:** which hosted model gives the best balance of speed and accuracy for finding elements? Test two models on the same 10 cases before choosing.

12. Sources Consulted

- DEV Community comparison of Browser-Use, Stagehand, Steel, and Playwright MCP
- Stagehand documentation on how DOM processing and vision fallback work
- Browser-Use library documentation on indexed interactive elements
- PageGuide paper summary on DOM-grounded assistance with indexed elements
- UI-TARS paper on GUI grounding and Set-of-Mark training
- MM-Navigator paper on Set-of-Mark prompting for screen navigation
- Paper on auxiliary reasoning for GUI grounding (Mark-Grid scaffold)
- Recent benchmarks comparing Set-of-Mark label selection with coordinate prediction

13. Next Module

Module 3, **Self-Healing Ladder and Blocker Handling**, builds on the failure types and the private locator map

defined here.